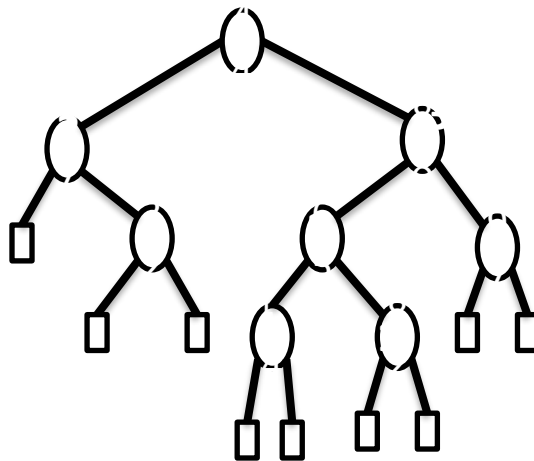


AVL Tree

CSE 203

Data Structures



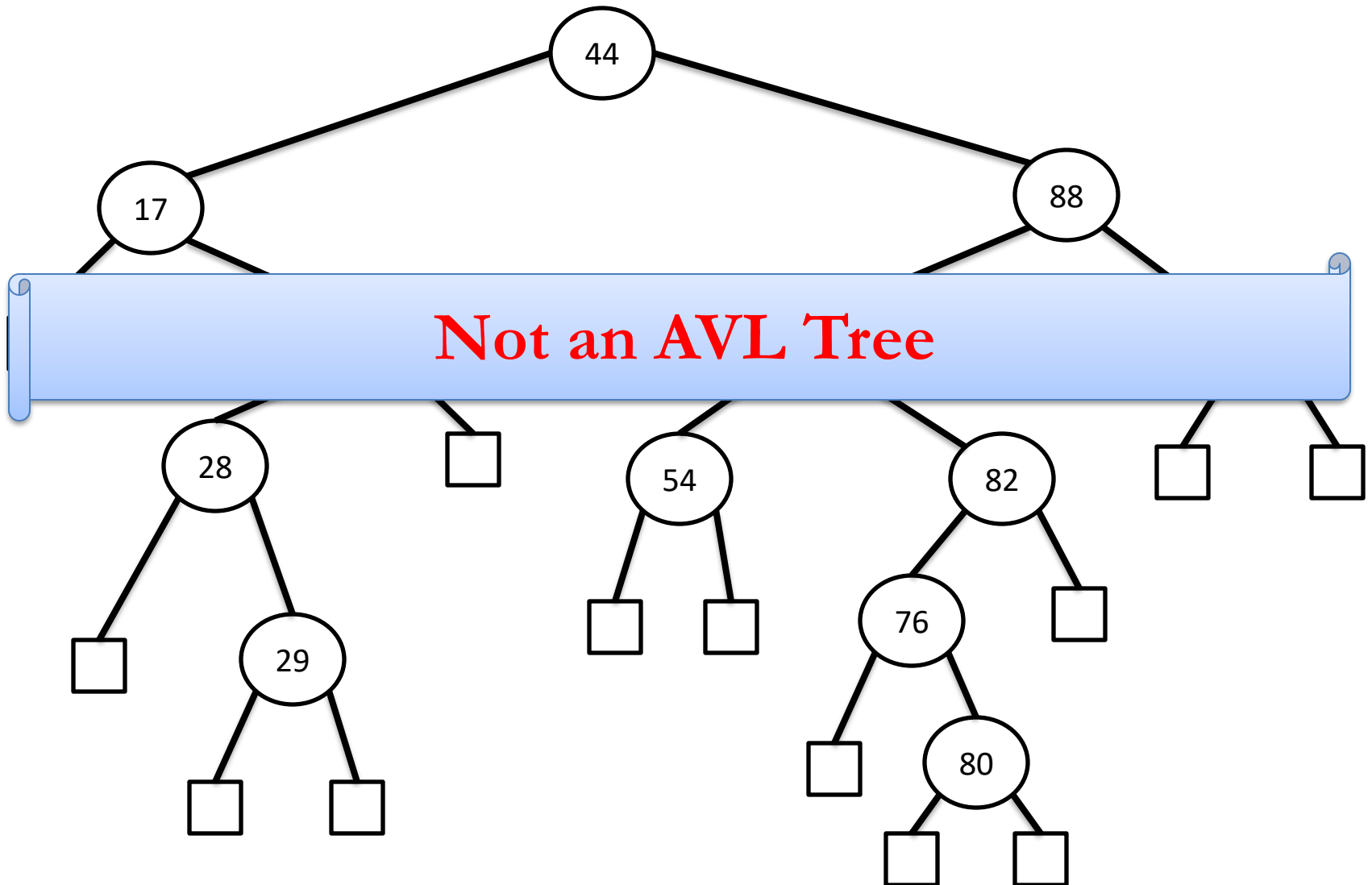
Height-Balance Property

- For every internal node v of T , the heights of the children of v can differ by at most 1.

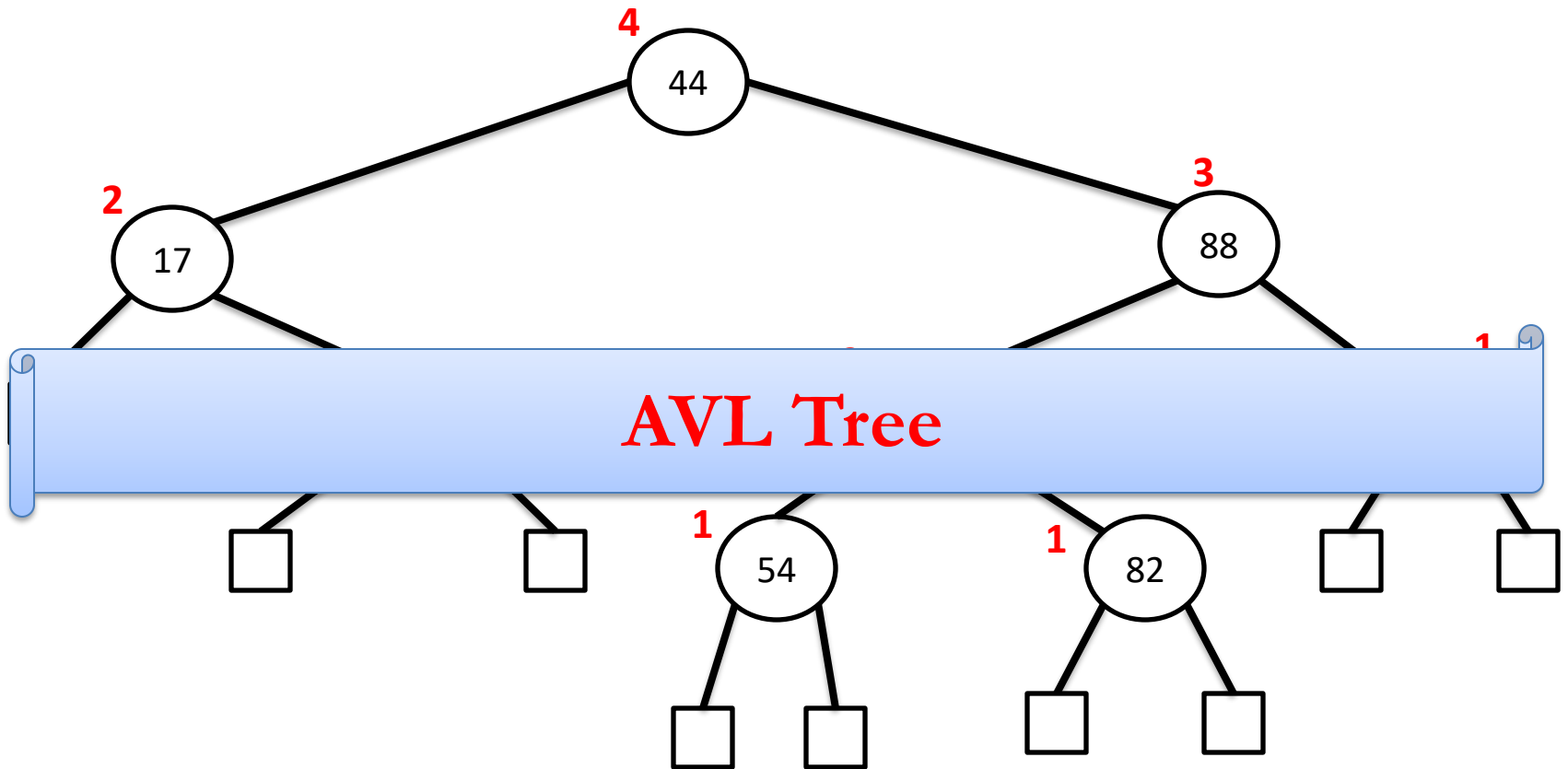
AVL Tree

- Any Binary Search Tree (BST) that satisfies the height-balance property is said to be *AVL tree*.
- Named after the Inventors –
 - Adel'son, Vel'skii and Lendis.

Binary Search Tree



Binary Search Tree



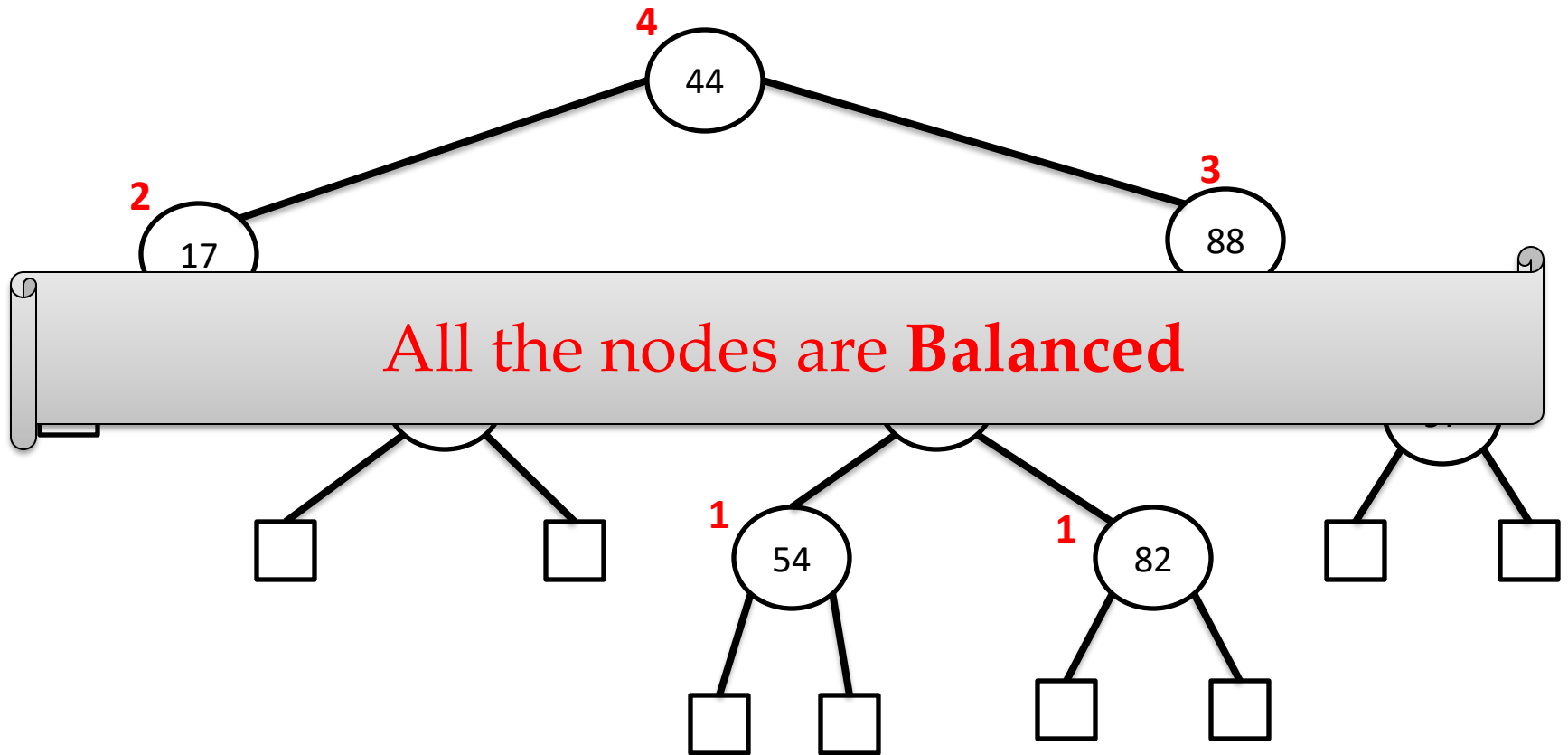
AVL Tree

- **Proposition:** The height of an AVL tree T storing n elements is $O(\log n)$

AVL Tree

INSERTION

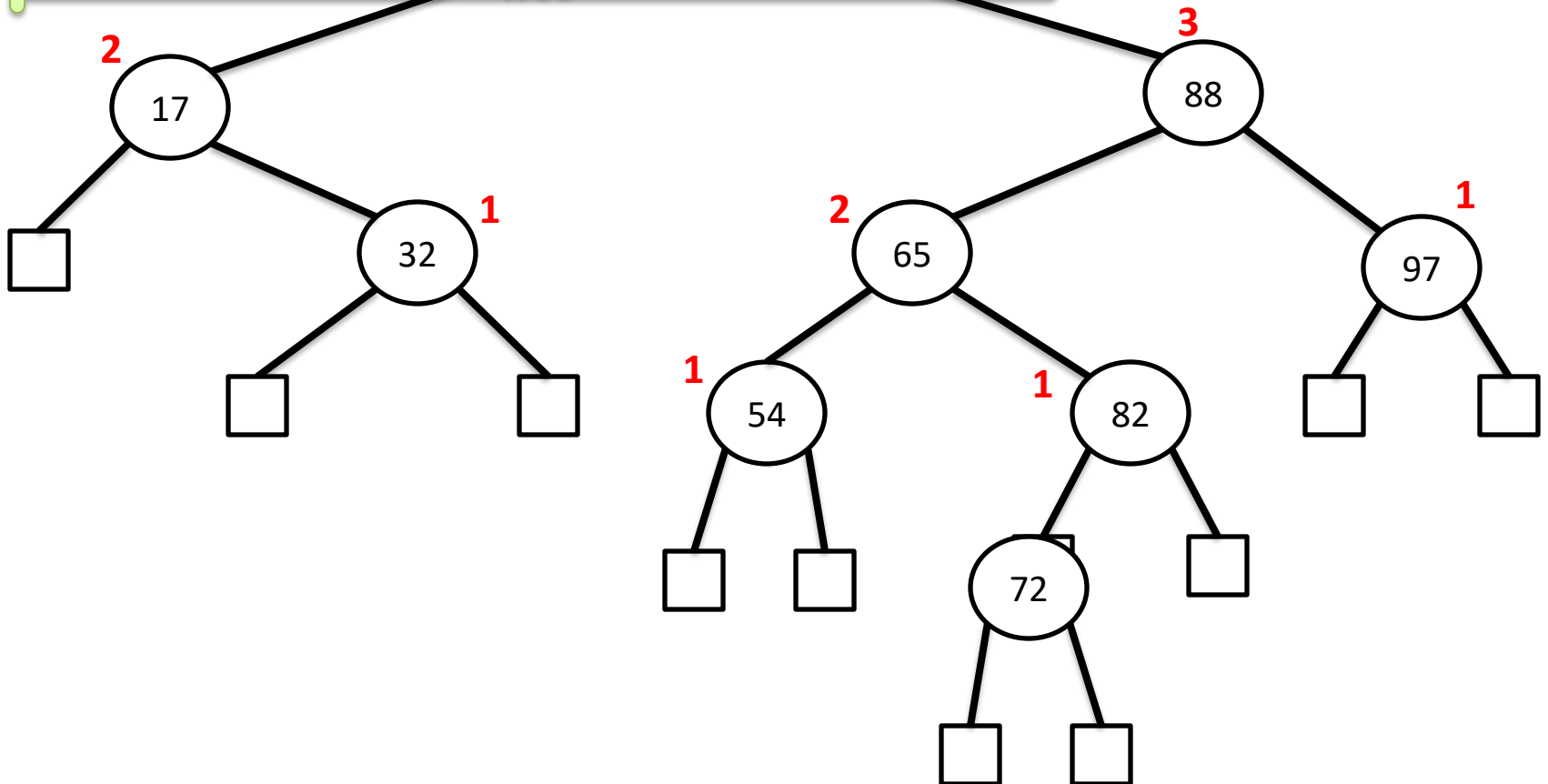
AVL Tree (Insertion)



AVL Tree (Insertion)

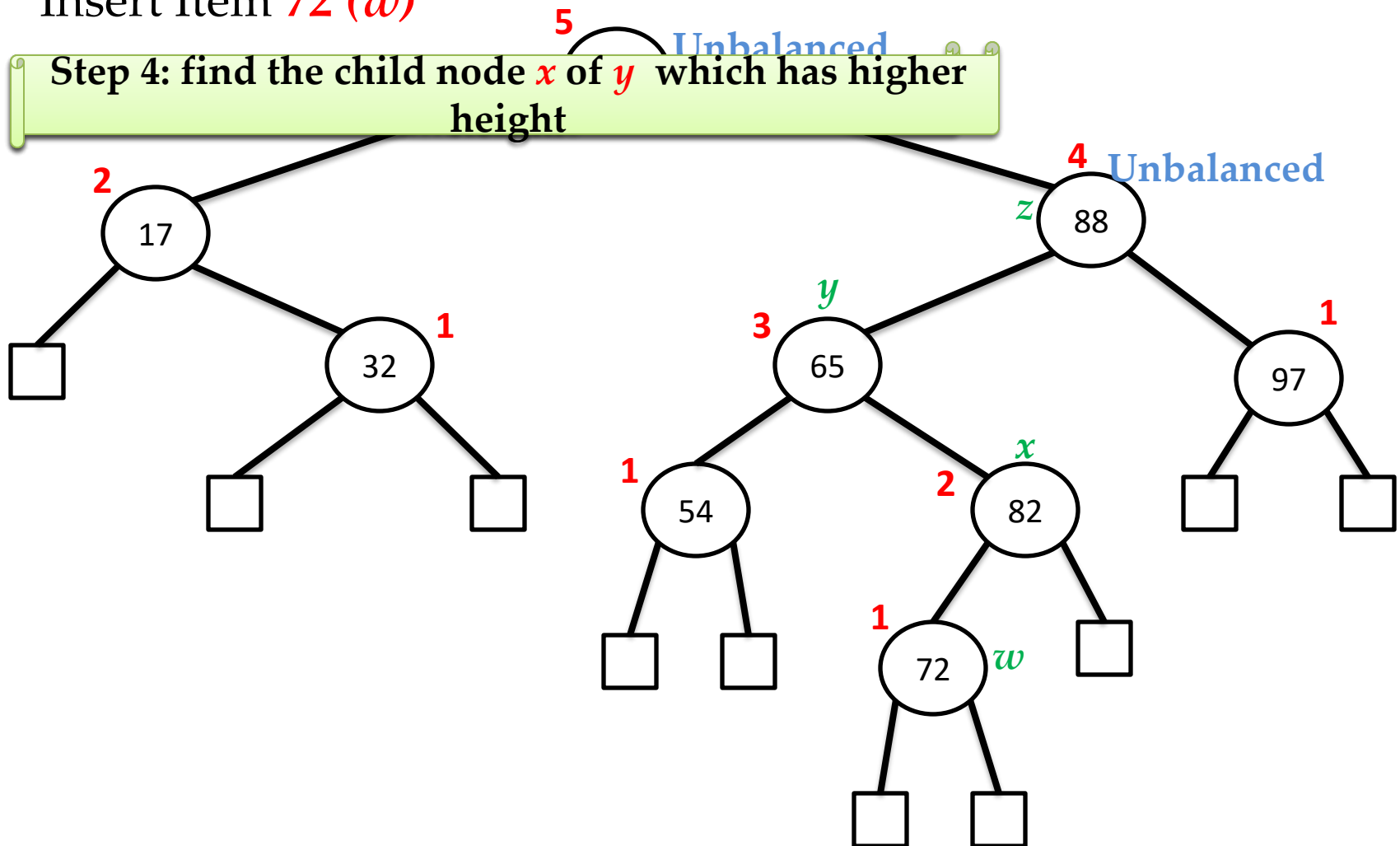
Insert Item **72**

Step 1: insertItem(**72**) as done in the **binary search tree**



AVL Tree (Insertion)

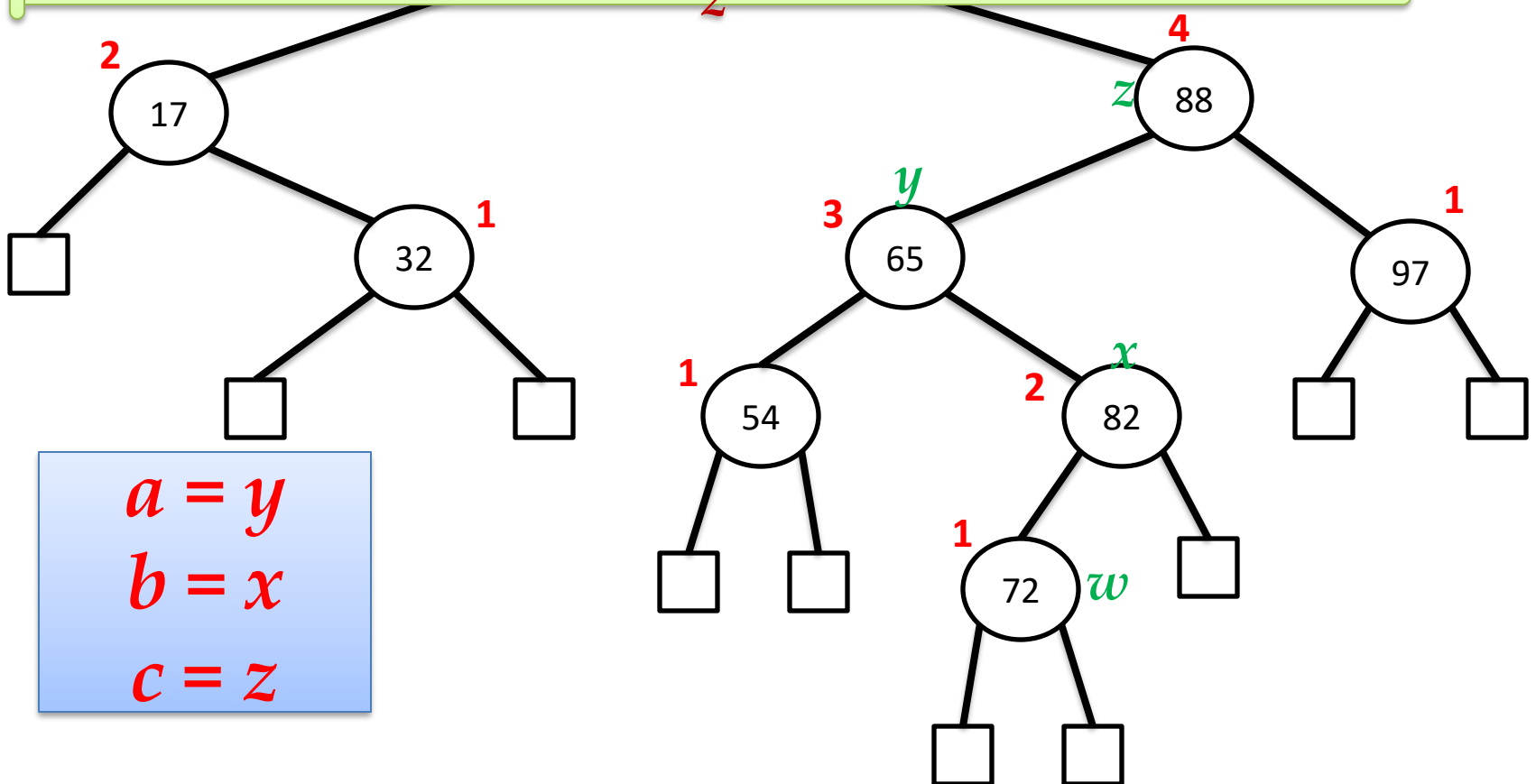
Insert Item **72** (*w*)



AVL Tree (Insertion)

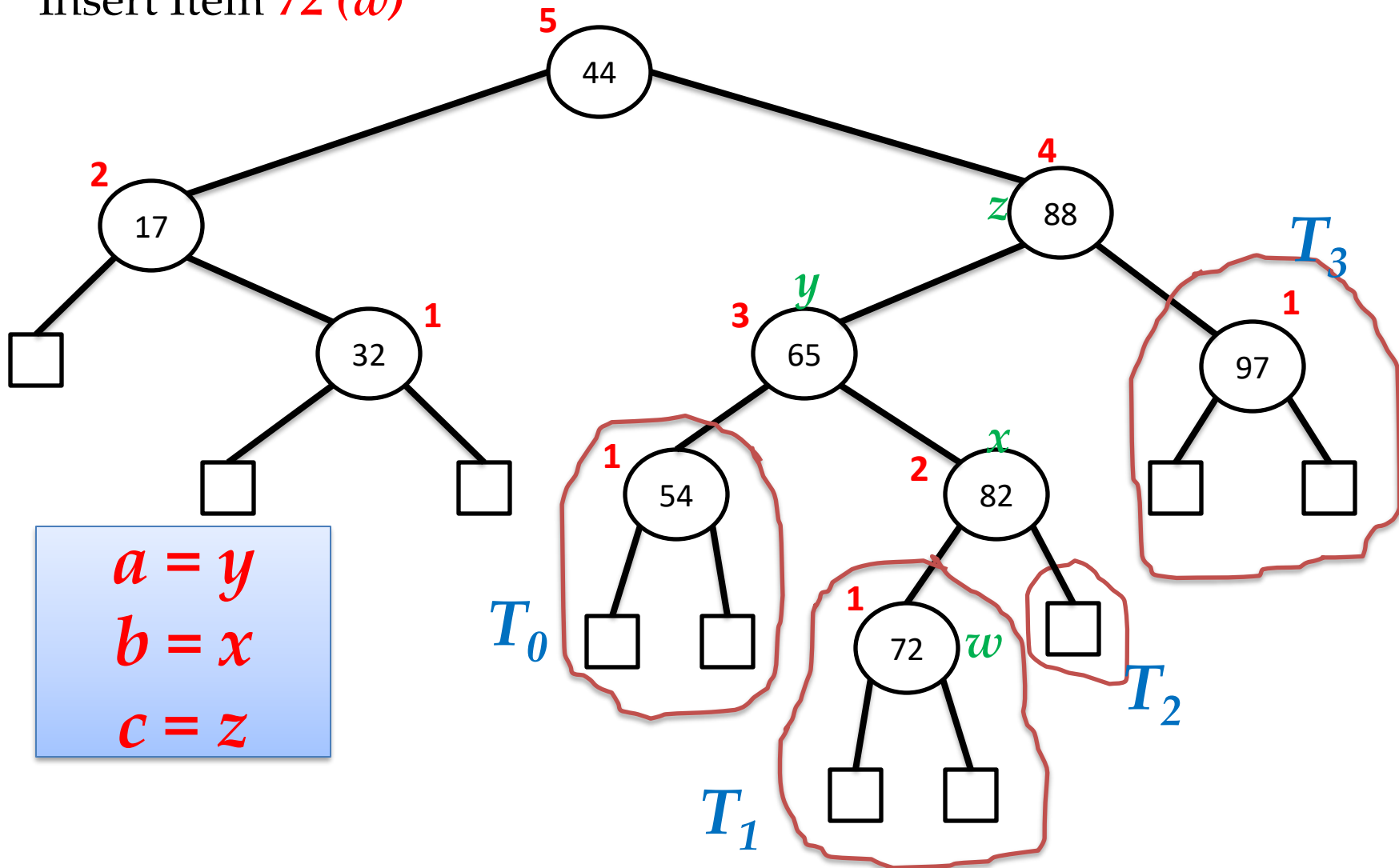
Insert Item **72** (*w*)

Step 5: **a, b, c** is the **inorder** listing of nodes **x, y** and **z**

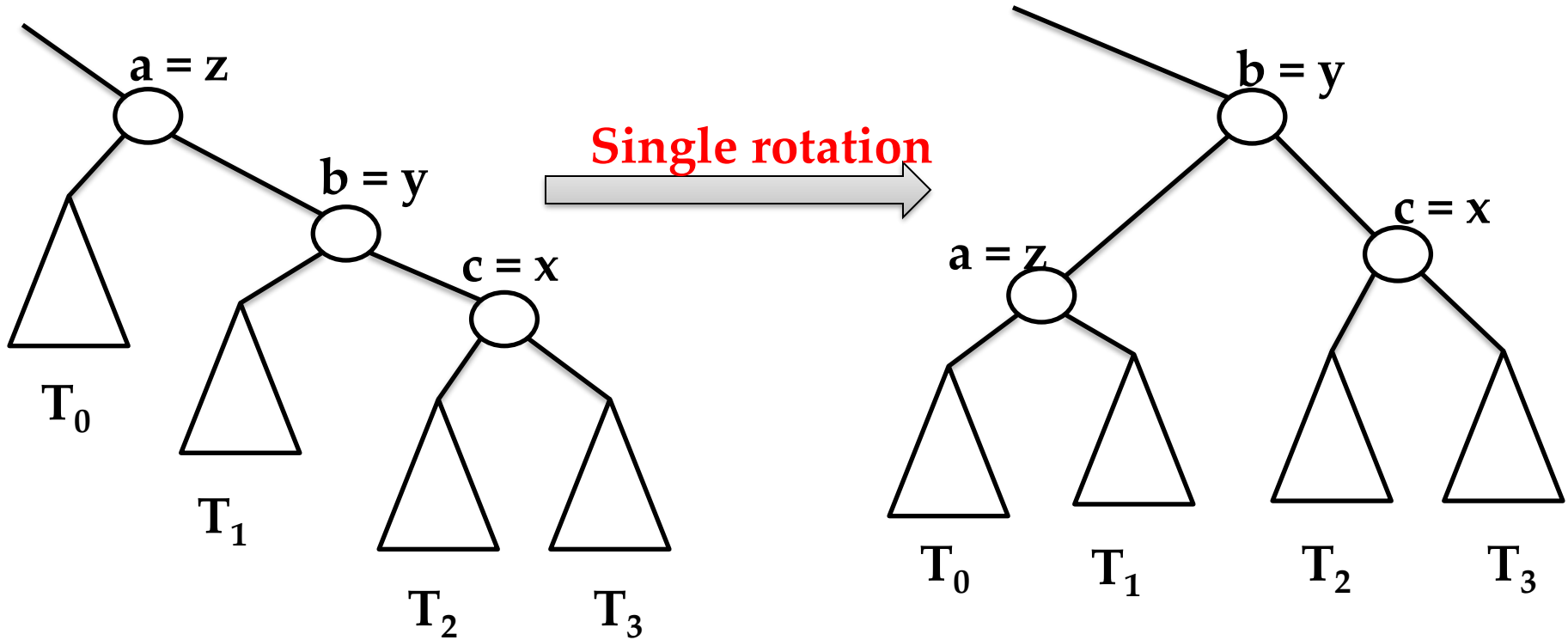


AVL Tree (Insertion)

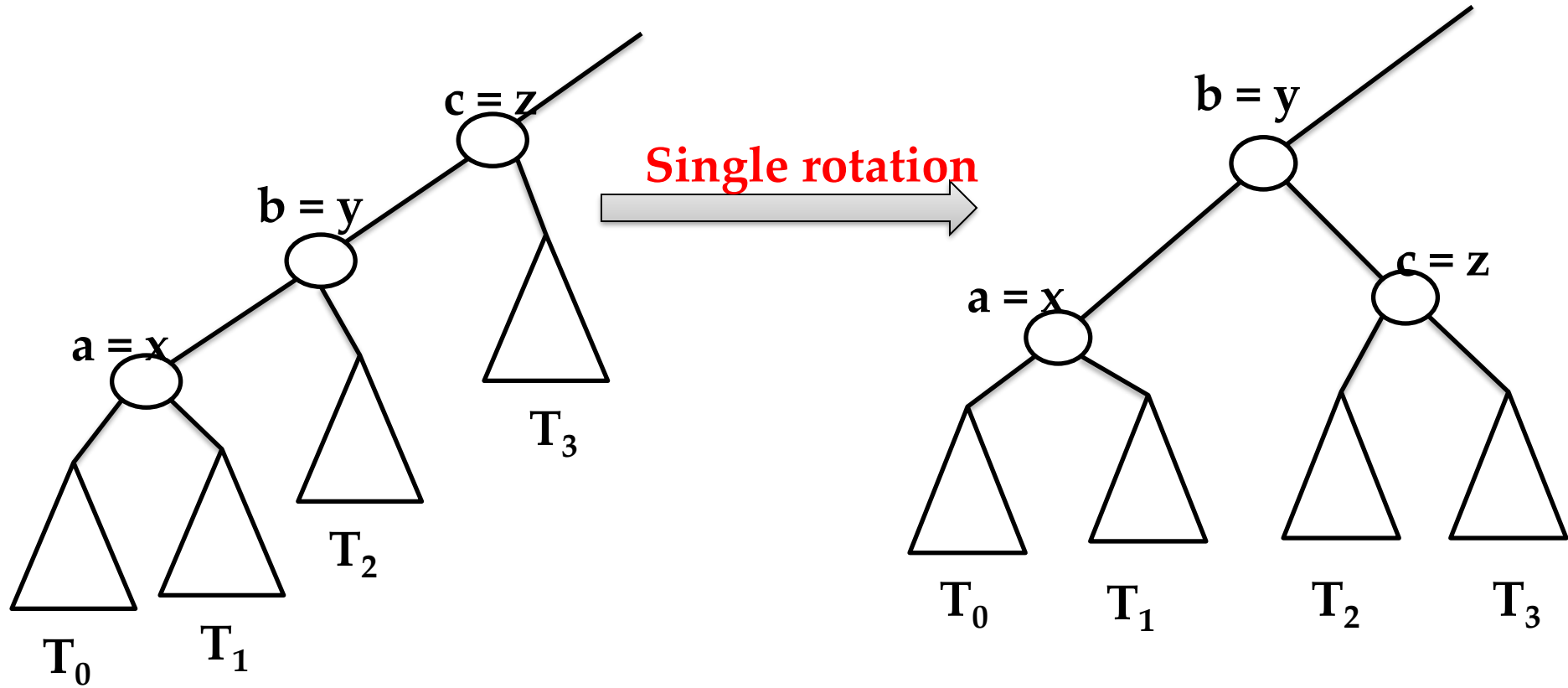
Insert Item **72** (*w*)



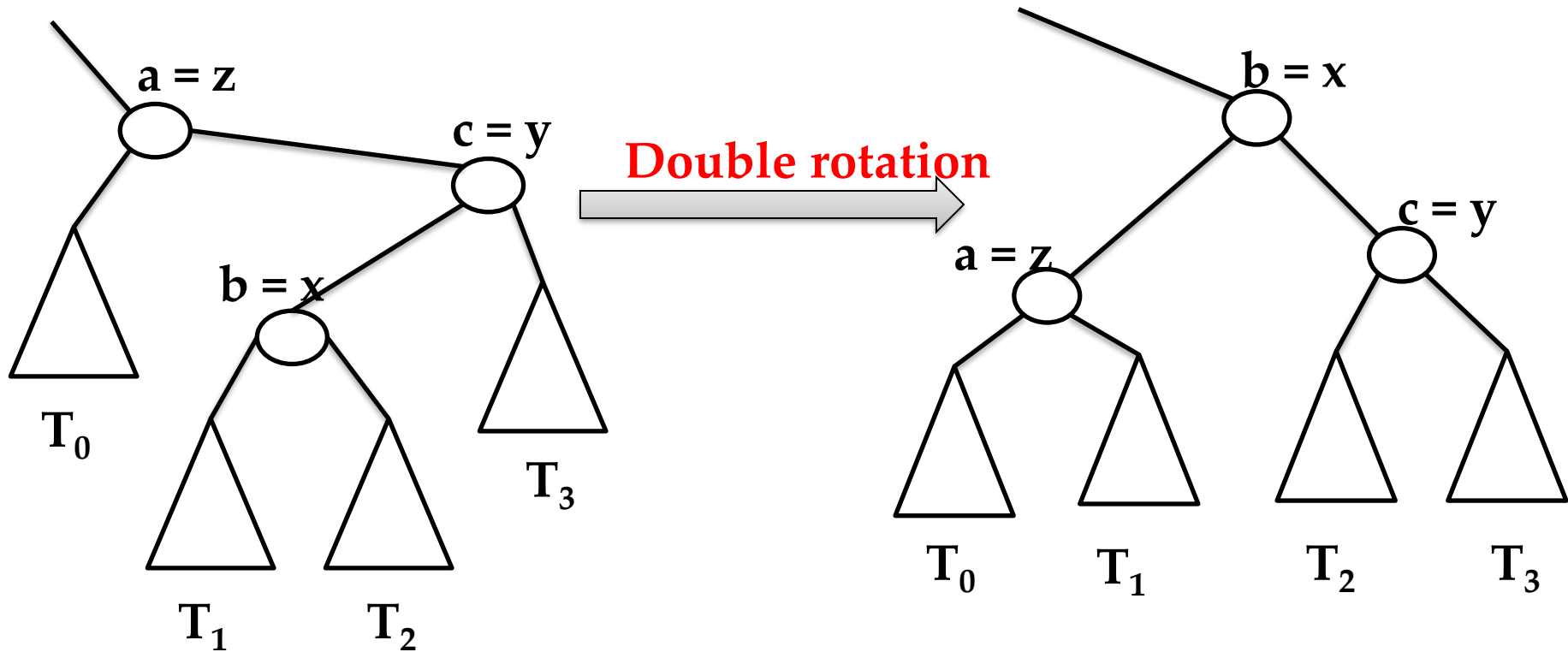
AVL Tree (Insertion)



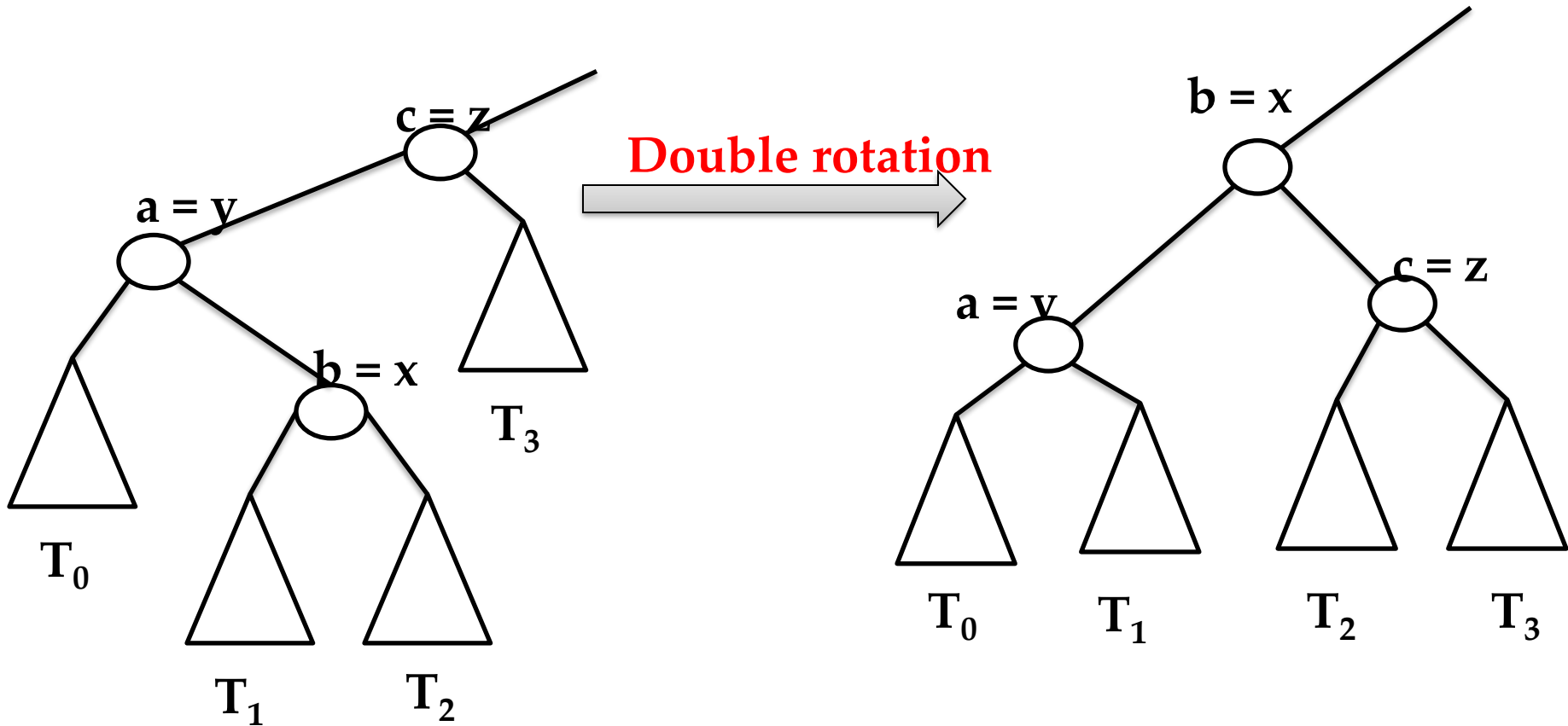
AVL Tree (Insertion)



AVL Tree (Insertion)

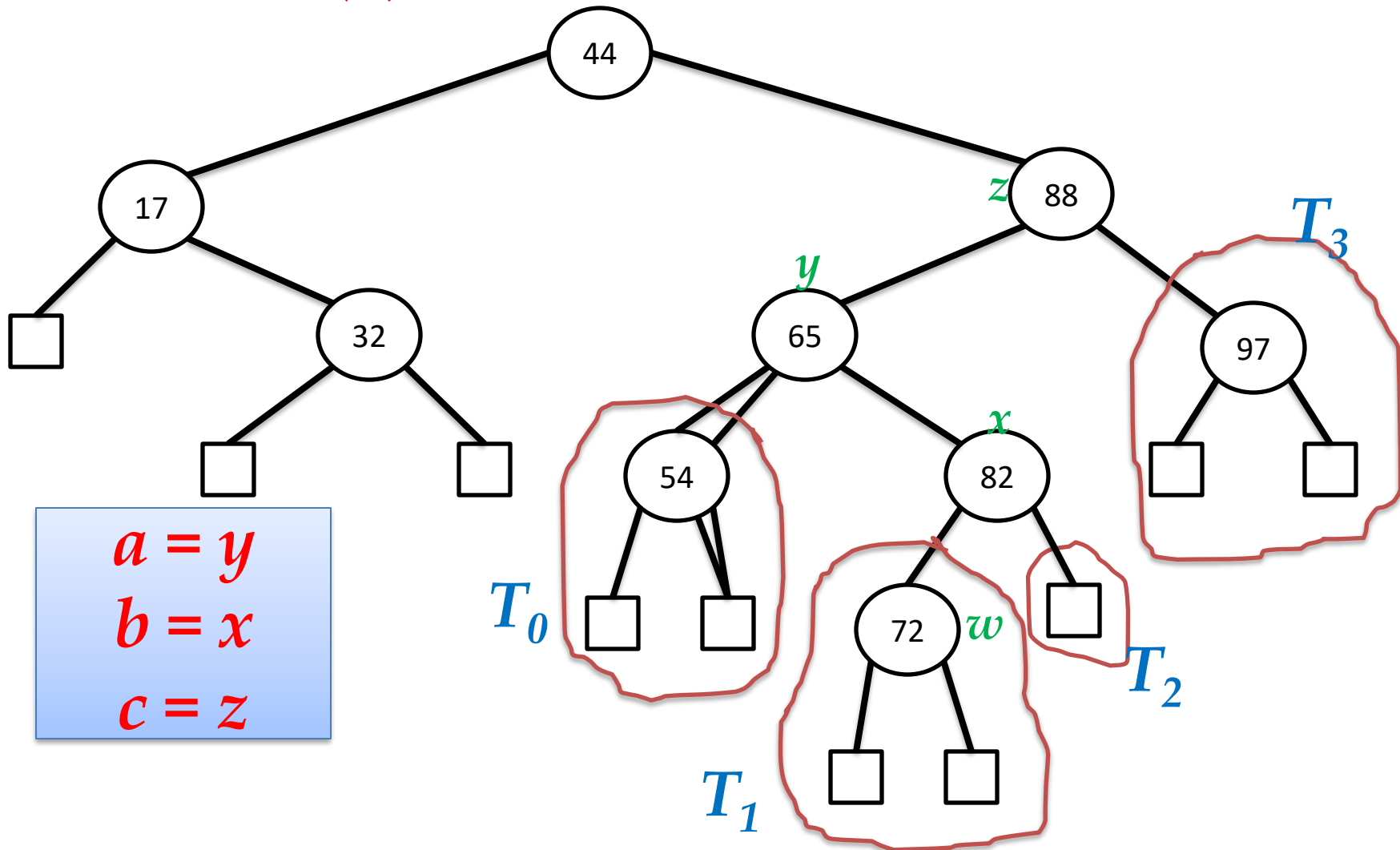


AVL Tree (Insertion)



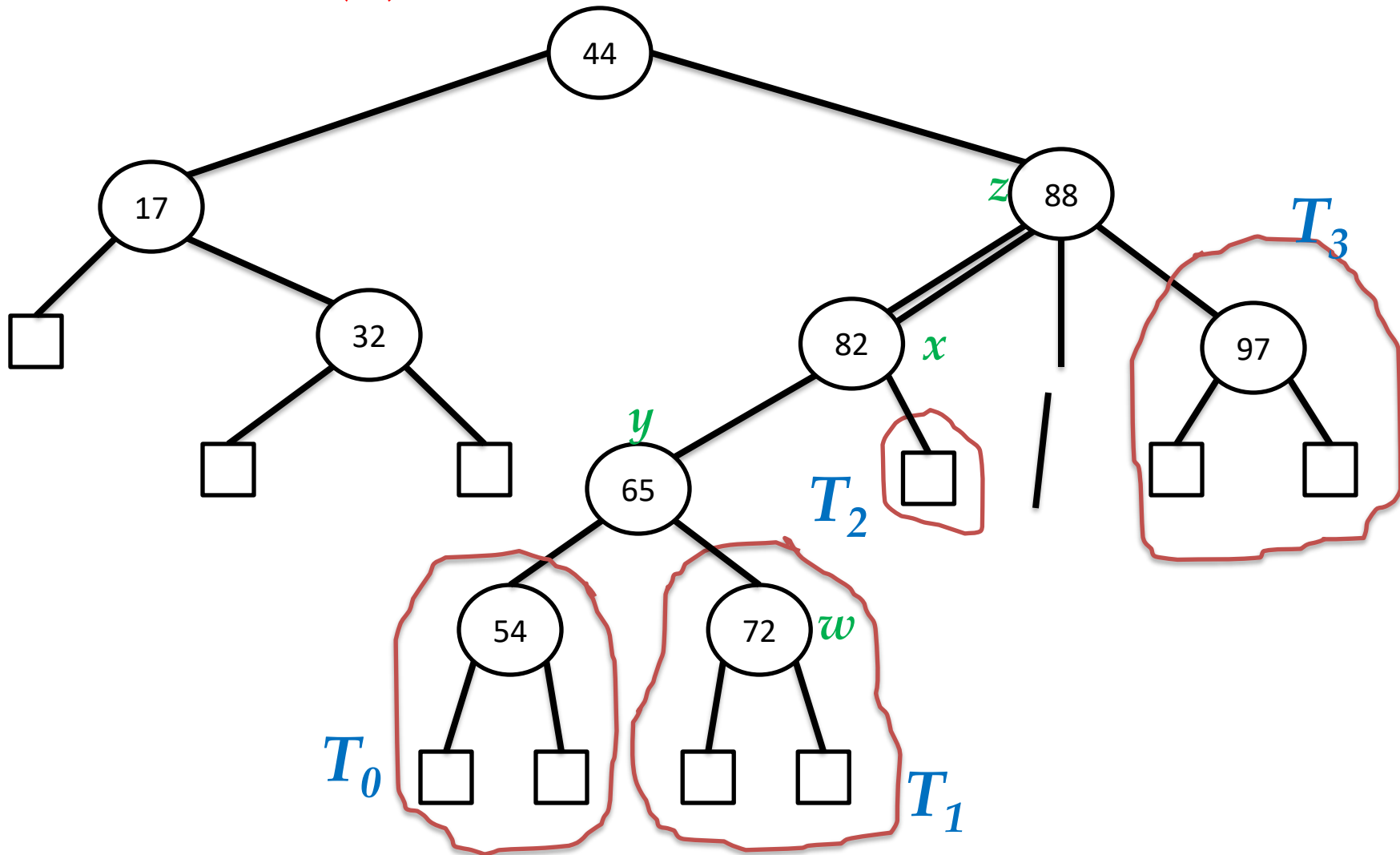
AVL Tree (Insertion)

Insert Item **72** (*w*)



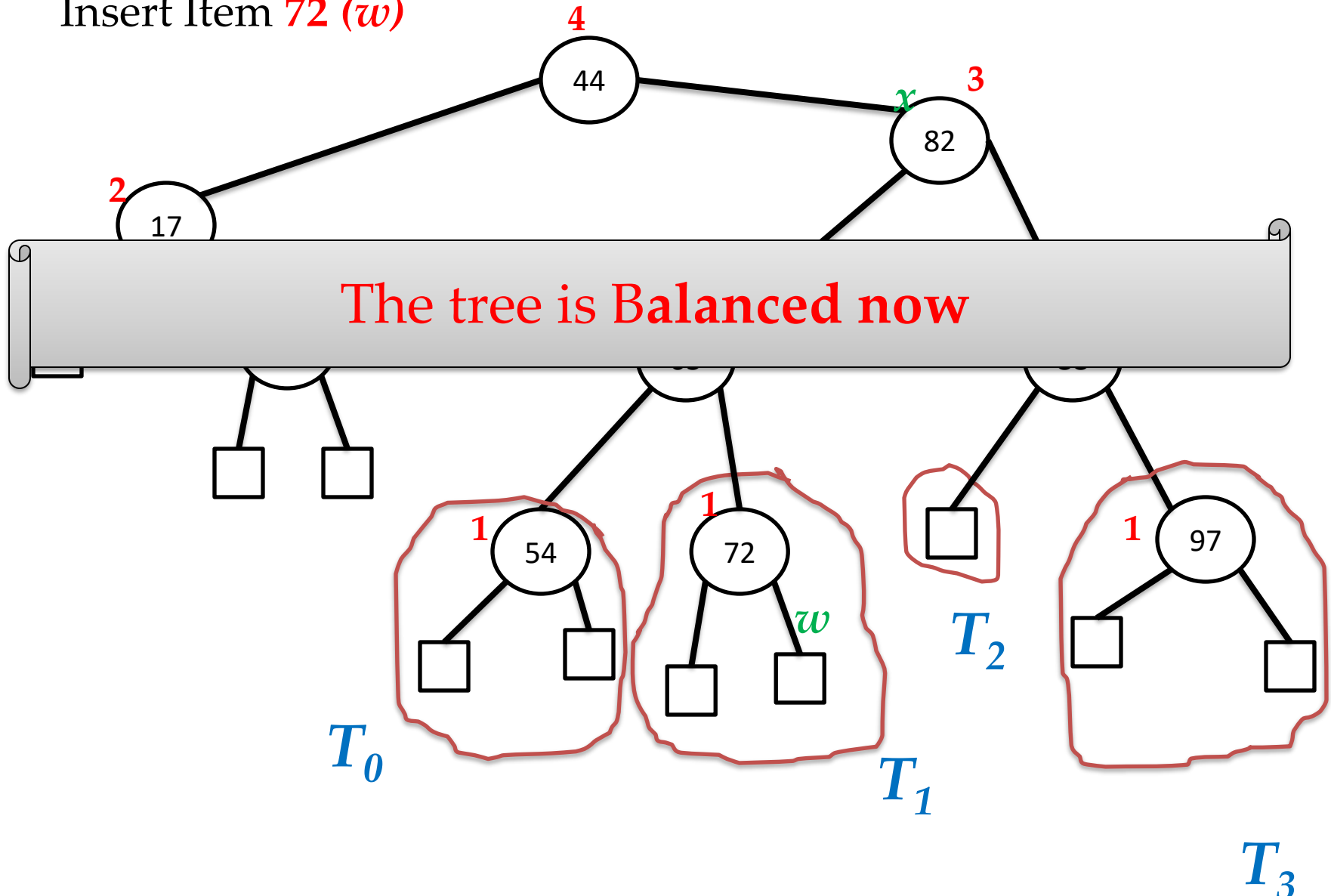
AVL Tree (Insertion)

Insert Item **72** (*w*)



AVL Tree (Insertion)

Insert Item **72** (*w*)



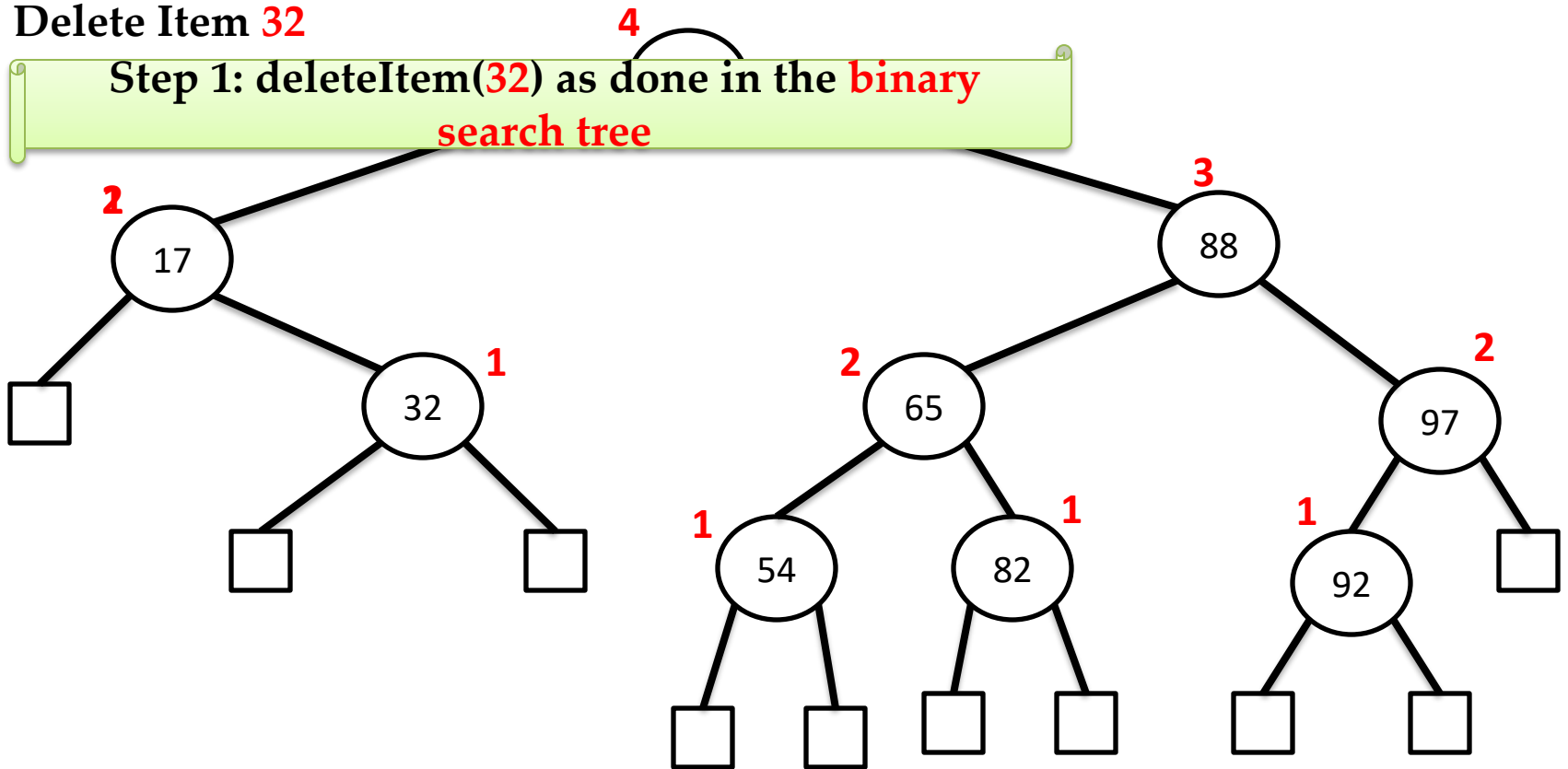
AVL Tree

DELETION

AVL Tree (Deletion)

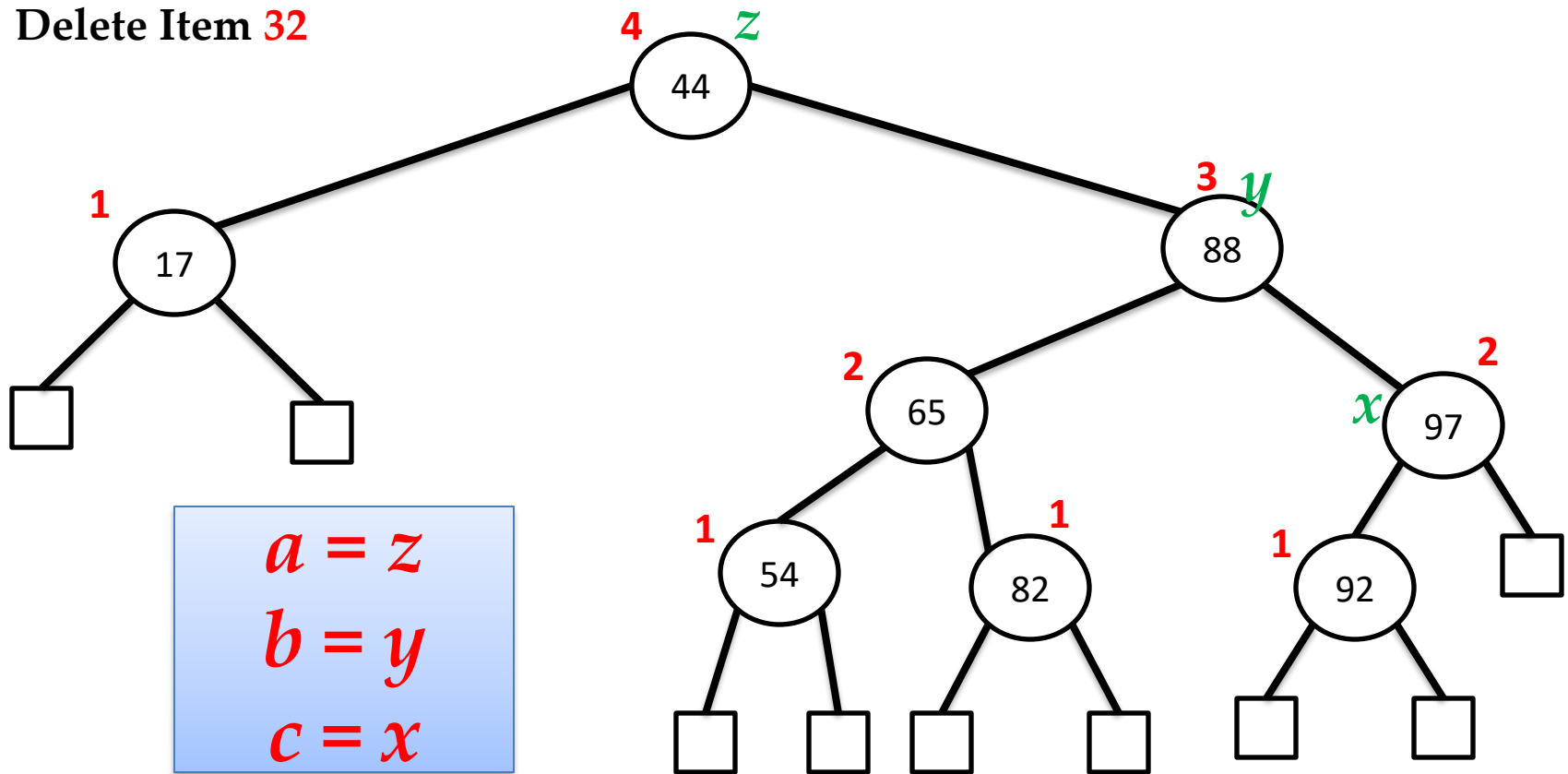
Delete Item **32**

Step 1: deleteItem(**32**) as done in the **binary**
search tree



AVL Tree (Deletion)

Delete Item **32**



$a = z$
 $b = y$
 $c = x$

After a **single rotation**

AVL Tree (Deletion)

